EL_5th_Sem — Engineering Overview

Microcontroller, Powering, Development Environment, Procurement, and CAN-FD Monitoring
Document status: technical study guide and implementation plan
Repository: https://github.com/Chirag-R-Karanth/EL_5th_Sem
Prepared: 23 October 2026

> **Scope note:** The repository TODO names five work items but does not identify a target MCU, source supply voltage, connector pinout, electrical loads, network bit rates, or monitored target. The reference directory contains `Architecture Overview.png` and `Flowchart.png`, but those binary diagrams were not readable through the available text-only repository connector. This guide explains the engineering decisions and gives a safe bring-up plan; it does not pretend unknown hardware choices are finalized. Confirm device-specific details against the actual schematic, datasheet, and reference manual before powering hardware.

Contents

1. System goal and high-level architecture
2. Microcontroller: what it does and how to choose one
3. Power architecture and safe bring-up
4. Development environment and firmware workflow
5. Vendors and procurement strategy
6. CAN-FD from fundamentals to implementation
7. Cross-monitoring and listen-only operation
8. Bench validation, failure diagnosis, and test plan
9. Decisions still required
10. Reference material
11. Quick review sheet
12. TODO coverage status

1. System goal and high-level architecture

The TODO describes a controller-based system that needs power, firmware development, procurement, and CAN-FD communication for “cross monitoring and listening.” The sensible first model is a CAN-FD node connected to an existing bus, either as a passive observer or as a second controller that exchanges test messages with another node.

Typical data path:

```
Power input -> protection and regulators -> microcontroller and
debug interface
Microcontroller CAN-FD controller -> CAN transceiver -> CANH/CANL
bus
Bus traffic -> transceiver receiver -> CAN-FD controller ->
firmware filters / receive FIFO -> timestamped log or diagnostic
output
```

Three distinct layers must not be confused:

- **CAN-FD protocol controller:** creates and parses frames, handles arbitration, bit timing, CRC, acknowledgements, error counters, and receive/transmit buffers. It may be integrated inside the MCU or implemented in an external controller.
- **CAN transceiver:** converts the controller’s logic-level TX/RX signals into the differential electrical signals on CANH and CANL. A protocol controller cannot directly drive the physical bus.
- **Firmware/application:** configures the peripheral, defines filters and message identifiers, handles received frames, timestamps them, and applies the project’s monitoring logic.

A fourth layer is the power system. Firmware is irrelevant if rails are noisy, undervoltage, sequenced incorrectly, or incompatible with the selected MCU/transceiver.

Recommended stages:

1. Identify exact hardware and constraints.
2. Power the board safely and verify every rail.
3. Build and flash a minimal firmware image.
4. Prove MCU-to-transceiver and CAN-FD operation using two known-good nodes.
5. Enable passive/listen-only monitoring on the target network.
6. Validate timestamps, filters, bus loading, error handling, and recovery.

2. Study the microcontroller

2.1 What the MCU contributes

A microcontroller combines a processor core, program memory, RAM, peripheral blocks, clocking, interrupt logic, and pins in one device. The core executes the program; flash stores firmware; RAM holds temporary state; peripherals handle timing-sensitive tasks such as CAN messaging without requiring the CPU to bit-bang every edge.

For CAN-FD work, the most consequential selection question is whether the MCU contains a **CAN-FD controller**, not merely a Classical CAN controller. A CAN 2.0-only peripheral cannot be made CAN-FD-capable by a firmware update alone. If it lacks integrated CAN-FD, use a supported external CAN-FD controller plus a transceiver, accepting added SPI traffic, interrupt handling, and integration complexity.

2.2 Selection checklist

- **CAN peripheral:** integrated CAN-FD/FDCAN/FlexCAN capability; number of independent controllers; receive FIFO depth; transmit queue/mailbox limits; hardware filters; timestamp support.
- **Bit timing:** separate arbitration and data-phase timing, bit-rate switching (BRS), and support for target data rates.
- **Clock:** stable source and tolerance suitable for selected CAN timing; confirm peripheral clock limits in the reference manual.
- **Compute and memory:** sufficient CPU time for receive bursts, logging, diagnostics, and application work; adequate RAM for queues and buffers.
- **Electrical interface:** I/O voltage compatibility with transceiver TXD/RXD/VIO pins; available pins and alternate-function mapping.
- **Debug:** accessible SWD/JTAG or vendor debug port, boot mode, and recovery path.
- **Power:** allowed supply range, current budget, low-power requirements, and regulator requirements.
- **Lifecycle and support:** datasheet/reference manual, errata, SDK maturity, example firmware, long-term availability, and board cost.
- **Environmental/reliability needs:** temperature range, ESD/transient exposure, and safety or qualification needs, if any.

Do not select an MCU by headline CPU MHz alone. CAN-FD reliability depends heavily on clock configuration, peripheral implementation, transceiver choice, PCB layout, bus topology, and correct bit timing.

2.3 Integrated controller or external controller?

Option A — MCU with integrated CAN-FD: the MCU peripheral connects directly to a CAN-FD transceiver through TX and RX signals. Usually this is the cleaner architecture: fewer components, lower latency, fewer SPI transactions, and simpler firmware.

Option B — MCU without integrated CAN-FD: an external CAN-FD controller communicates with the MCU over SPI and connects to a transceiver. This can preserve an existing MCU choice, but adds parts, clock requirements where applicable, interrupt lines, driver dependencies, and throughput constraints over SPI.

For a new design, prefer integrated CAN-FD if it meets the project's MCU needs. For an existing host MCU that cannot be changed, consider an external controller only after validating its driver, SPI throughput, interrupt behavior, and compatibility.

2.4 Read the right documents in order

1. **Datasheet:** supply ranges, absolute maximum ratings, pinout, electrical characteristics, package, peripheral availability.
2. **Reference manual:** clock tree, GPIO alternate functions, CAN-FD register model, filters, FIFOs, error status, interrupts, and timing registers.
3. **Errata:** silicon-specific bugs or workarounds. Check these before blaming firmware for unexpected peripheral behavior.
4. **SDK/manual examples:** build system, startup code, linker script, clock initialization, and vendor CAN examples.
5. **Board schematic:** actual pins, oscillator, termination jumpers, regulators, connectors, transceiver standby pin, and debug access.

2.5 First MCU bring-up sequence

- Identify the exact part number and board revision; related boards can differ in peripheral counts and pin mapping.
- Verify supply rails, reset state, and clock source before debugging CAN.
- Connect the debugger and flash the smallest supported example.
- Confirm a heartbeat using a GPIO or UART. This validates build, flash, reset, clock, and runtime.
- Initialize CAN-FD using vendor-supported code; read back configuration and status registers if practical.

- Start with a low-risk bench network with two known-good nodes and short twisted-pair cable.
- Only after the bench test passes, connect to the real monitoring network.

3. Powering

3.1 Why power design deserves separate treatment

Powering is not just connecting a battery to a board. Input supply, protection, conversion efficiency, regulator stability, startup transient, grounding, and current peaks determine whether the MCU and transceiver behave predictably. CAN errors can be caused by supply droop or resets and misdiagnosed as protocol bugs.

The repository does not state input voltage or load current, so exact regulator, fuse, or resistor values cannot responsibly be specified. Derive values from the actual supply, datasheets, and measured loads.

3.2 Map the power tree

Write down every rail and every load before choosing components. Example only:

```
Input connector -> reverse-polarity / transient protection (if
required)
                -> input filtering -> regulator(s)
                -> MCU core / MCU I/O / transceiver / debug
accessories
```

The real tree may differ. Some MCUs use one external rail but generate internal core rails; others need several supplies or special analog, backup, or USB pins. Follow the selected MCU datasheet and reference board schematic.

Create a rail budget containing rail name, nominal voltage, minimum/maximum allowed voltage at each load, steady-state and peak current, startup/inrush behavior, regulator rating and thermal dissipation, required decoupling, and a test point with measured voltage.

For a linear regulator, approximate dissipation as:

```
P_loss = (V_in - V_out) × I_out
```

A large voltage drop at meaningful current can overheat a linear regulator. A switching regulator is often more efficient but adds layout, switching-noise, and stability considerations. Choose from power loss, noise sensitivity, load profile, efficiency, thermal margin, and complexity—not habit.

3.3 Protection and decoupling

Depending on supply source and environment, consider reverse-polarity protection, fuse/current limiting, transient-voltage suppression, input filtering, and brownout monitoring. The correct circuit depends on whether input is USB, a bench supply, a vehicle battery, or another source. Automotive battery inputs need a substantially different protection strategy from a clean bench supply.

Place local bypass capacitors at IC supply pins using the manufacturer's recommended values and placement. Bulk capacitance near a connector/regulator can support lower-frequency load changes; small ceramic capacitors close to the IC reduce high-frequency impedance. The right nominal value poorly placed is not equivalent to effective local decoupling.

Keep high-current/switching return paths from corrupting sensitive references or the CAN interface. Follow good PCB layout practices: short regulator switching loops, correct ground returns, controlled routing around the oscillator, and compact transceiver/bus-protection routing.

3.4 Sequential power-up test

1. Inspect for shorts, polarity errors, solder bridges, and incorrect jumper positions.
2. Use a current-limited bench supply at the correct documented voltage. Set a conservative current limit based on expected startup current.
3. Power the board with external loads and CAN bus disconnected initially.
4. Measure all important rails and input current; use an oscilloscope for startup, ripple, and resets if possible.
5. Check regulator and IC temperature. Disconnect power immediately if current, voltage, or temperature is abnormal.
6. Confirm stable reset release and heartbeat firmware.
7. Connect the transceiver and test network only after local power is stable.

8. Test restart and brownout behavior; recovery should be predictable and should not leave the transceiver in an unintended state.

3.5 Power-related failure symptoms

- **Unusually high current:** short, wrong polarity, solder fault, damaged IC, or regulator instability.
- **Repeated MCU resets:** rail droop, brownout threshold, watchdog, clock startup, or firmware fault.
- **CAN fails only during transmission:** transceiver supply droop, grounding/EMI, bit timing, bus wiring, or termination.
- **Hot regulator:** excessive voltage drop/current, oscillation, insufficient thermal area, or wrong component.
- **Debugger not detected:** check power, ground reference, reset, boot straps, debug wiring, and target voltage before blaming firmware.

4. Development environment

4.1 A reproducible toolchain, not just an IDE

A working embedded toolchain includes compiler, assembler, linker, debugger, programmer/flasher, SDK or HAL, build system, serial/logging tools, and version control. If another machine cannot reproduce the build, the project is fragile.

Typical components:

- **Toolchain:** vendor-supported GCC/LLVM or proprietary compiler for the selected MCU.
- **Build system:** vendor IDE project or a maintained CMake/Make flow.
- **Debugging:** GDB with a supported probe/debug server, or vendor debugger.
- **SDK/HAL:** clock/startup code, peripheral headers, CAN-FD driver, linker script, and device packs.
- **Communication tools:** serial terminal, CAN interface tool, frame logger, and a second CAN-FD node/interface.
- **Version control:** Git for source, configuration, scripts, test notes, and README. Do not commit generated build folders or secrets.

Use the vendor-supported toolchain first to get a working build. Once a known-good board example works, decide whether moving to CMake or another workflow adds enough value. Rewriting the build system before validating hardware wastes time.

4.2 Suggested project structure

```
firmware/
  src/          application and project-owned drivers
  include/      public project headers
  board/        board-specific pin and clock configuration
  tests/        unit/host-side tests where practical
  config/       documented build and CAN parameters
scripts/        repeatable build/flash/log helpers
hardware/       schematics, PCB files, pin tables, BOM
docs/           wiring, bring-up, protocol matrix, decisions
README.md       prerequisites and exact build/run steps
```

Do not copy vendor SDK code into project files without recording origin and license. Keep application policy separate from MCU register-level or vendor-driver details where practical.

4.3 Firmware layering

Separate hardware initialization (clocks, GPIO, watchdog, power signals); CAN driver/configuration (nominal timing, data timing, filters, RX/TX queues, interrupts, error counters); transport/logging (timestamps, frame formatting, dropped-frame counters, bounded buffers); application logic (which identifiers and signals matter); and diagnostics (startup self-check, peripheral state, bus-off recovery policy, and firmware version/build ID).

Avoid lengthy logging or parsing inside a CAN interrupt. The interrupt should capture or enqueue the minimum data needed and signal a task/loop. Otherwise, receive bursts can cause dropped frames or increased latency. If using an RTOS, define ownership and synchronization for receive queues. If using a superloop, ensure it drains traffic fast enough and buffering is bounded.

4.4 Reproducible setup checklist

- Record MCU model, board revision, SDK/compiler versions, probe, operating system, and CAN interface.
- Pin known-good tool versions or document installation commands.
- Build from a clean checkout with one documented command.

- Flash and verify a heartbeat program.
- Store CAN timing in one named configuration, not scattered magic numbers.
- Emit firmware version or commit ID in diagnostic output.
- Document target recovery and restoration of boot/debug access.

5. Vendors and procurement

5.1 Buy by requirements

The TODO says “Vendors to pocure” (likely procurement). First make a bill of materials and define mandatory constraints. Exact part numbers cannot be finalized until the MCU, supply, number of CAN channels, target bitrate, environmental conditions, connector, and availability constraints are known.

Typical procurement categories:

- MCU or evaluation board, preferably one with integrated CAN-FD for a new platform.
- CAN-FD transceiver matched to supply voltage, logic interface, data rate, standby/wake behavior, package, and environment.
- CAN interface/analyzer for PC logging and transmission, ideally with explicit listen-only mode and CAN-FD support.
- Debug probe and cables compatible with the MCU.
- Power components: regulator(s), protection, capacitors, connectors, fuses, and test points.
- Bus materials: twisted-pair cable, connectors, appropriate termination, optional ESD/transient protection.
- Test equipment: current-limited bench supply, multimeter, and oscilloscope/logic analyzer as appropriate.

5.2 Vendor families to investigate

These are starting points, not an approved BOM:

- **STMicroelectronics:** STM32 families with FDCAN and a Cube/IDE/application-note ecosystem.
- **NXP:** S32K automotive MCU families with CAN-FD peripherals and automotive-oriented software/tooling.
- **Microchip:** PIC/SAM devices with integrated CAN-FD; MCP2517FD/MCP2518FD external controller families for suitable host MCUs.

- **Texas Instruments:** CAN-FD transceivers and external-controller/transceiver solutions such as TCAN4550.

Vendor presence does not guarantee a specific part is in stock or suitable. Verify current datasheets, errata, authorized distributor availability, minimum order quantities, lifecycle status, and delivery estimates before procurement.

5.3 Development board versus custom PCB

For the first proof of concept, use development boards and a known-good CAN-FD USB/interface if possible. Prove firmware and protocol operation before introducing PCB layout and power-integrity variables. A custom PCB is justified when pinout, connectors, power, transceiver, protection, mechanical dimensions, and test points are stable.

5.4 BOM columns

```
Part | Manufacturer | Exact part number | Function | Electrical constraints | Package | Quantity | Unit price | Source | Lead time | Lifecycle status | Alternate | Datasheet URL | Design notes
```

Choose alternates by electrical compatibility, not by vaguely similar names. For transceivers, confirm CAN-FD data rate, supply, common-mode range, standby logic, TXD/RXD behavior, fault protection, temperature grade, and intended bus environment.

6. CAN-FD: fundamentals to implementation

6.1 What CAN is

Controller Area Network (CAN) is a multi-master, message-oriented serial bus. Nodes place frames on a shared differential bus. Frames are identified by a message identifier, which also participates in arbitration priority; it is not simply a destination address. Multiple nodes can receive the same frame and decide in firmware whether its contents matter.

CAN uses dominant and recessive logical bus states represented physically through the differential CANH/CANL pair. During arbitration, a node that transmits recessive but observes dominant

knows it lost arbitration and stops transmitting without corrupting the winning frame. This is **non-destructive arbitration** and lets higher-priority identifiers win access to the bus.

6.2 Why CAN-FD exists

Classical CAN carries up to 8 data bytes per frame. CAN-FD expands the payload to as many as 64 bytes and permits a higher data-phase bit rate while retaining a slower arbitration phase when bit-rate switching is enabled. Larger payloads reduce overhead for data-heavy messages. Actual usable bit rate depends on controllers, transceivers, clocking, wiring, topology, and network timing margins; a headline maximum is not a guarantee for the project wiring.

CAN-FD is defined under the ISO 11898-1 family of standards. Use the exact standard edition required by the application and check compatibility of the tools and devices on the network.

6.3 Arbitration and data phases

A CAN-FD frame starts at the nominal/arbitration bit rate so nodes can participate in arbitration. If BRS enables switching, the faster data bit rate is used for the relevant data portion. The controller switches timing at protocol-defined points and returns to nominal timing as required before acknowledgement/end portions. Hardware handles the protocol details, but firmware must configure correct nominal and data timing.

Key fields/concepts:

- **Identifier:** 11-bit base identifier or 29-bit extended identifier; lower numeric values generally win arbitration when compared in the same arbitration context.
- **FDF/EDL:** identifies CAN-FD frame format relative to Classical CAN.
- **BRS:** selects whether the frame switches to the configured data-phase bit rate.
- **ESI:** reflects the transmitting node's error state (error-active or error-passive).
- **DLC:** data length code. CAN-FD codes map to payload lengths up to 64 bytes; not every integer length from 9 through 64 is encoded directly.
- **CRC:** detects transmission corruption. CAN-FD uses stronger CRC handling than Classical CAN and defined bit-stuffing rules.

- **ACK:** a receiver that correctly receives a frame can acknowledge it at the protocol level. ACK does not mean the application accepted the message or acted on it.
- **Error counters and bus-off:** hardware tracks protocol errors and can enter bus-off after severe error accumulation. Firmware must observe and handle this state.

6.4 Bit timing

Bit timing is not only a bitrate number. A nominal bit time is divided into time quanta and segments, with a prescaler, sample point, and synchronization jump width. The data phase has its own timing settings for CAN-FD. The sample point must account for propagation delays across the network and transceiver/cable behavior.

Never copy timing values blindly from another board. Confirm the peripheral clock first, then use the vendor's timing calculator or documented method. Communicating nodes must agree on nominal bitrate, and BRS nodes must agree on data bitrate and compatible timing. Mismatches often produce error frames, retransmissions, or no usable traffic.

For the initial bench test, begin at conservative, well-supported bitrates from development-board examples. Increase only after the connection is stable. Target-network values must come from documentation or measurement—not guesswork.

6.5 Physical bus and termination

CANH and CANL form a differential pair. Use suitable twisted-pair wiring, keep stubs short, avoid star wiring unless designed for it, and follow transceiver/system guidance. High data rates reduce tolerance to long stubs and poor topology.

A properly designed linear CAN bus is normally terminated at its two physical ends with 120-ohm resistors, not at every node. With power removed and both terminators present, a resistance measurement across CANH and CANL often reads approximately 60 ohms because two 120-ohm resistors are in parallel. This is a sanity check, not a substitute for inspecting actual topology; split termination and disconnected nodes can change measurement conditions.

Do not add a new 120-ohm termination to an existing bus without confirming the node is at an end and termination is not already present. Extra termination can overload the driver; missing termination can cause reflections and data-phase errors. Fast CAN-FD data phases make signal integrity particularly important.

6.6 Controller versus transceiver

Integrated-controller MCU: MCU TX output goes to transceiver TXD; transceiver RXD returns to MCU RX input; transceiver CANH/CANL connect to the bus. Transceiver standby/enable pins must be in the correct state, and logic levels must be compatible.

External-controller MCU: MCU communicates with an external CAN-FD controller over SPI. That controller connects through its TX/RX interface to the physical-layer transceiver. This requires a suitable driver and adds complexity.

The controller handles frame-level protocol; the transceiver handles electrical-level transmission and reception. Confusing these roles leads to incorrect component selection.

6.7 Frames, identifiers, and filters

A monitoring application should decide whether to accept every identifier or only a known subset. Hardware filters reduce interrupt and CPU load, but overly restrictive filters can hide the frames needed for debugging. Start with receive-all or permissive filters on an isolated bench bus; narrow them after comparing logs against a reference analyzer.

For every monitored ID, document its standard/extended format, Classical CAN/CAN-FD expectation, payload length, expected BRS usage, period/trigger behavior if known, signal encoding/endianness/scaling if decoding is required, expected sender, validity checks, and handling of unknown IDs.

A frame log should record timestamp, bus/channel, identifier, frame format, BRS flag, payload length, payload bytes, error/status flags, and drop counters where supported. Treat timestamps as relative to a known clock and document whether timestamps are hardware or software timestamps.

7. Cross-monitoring and listen-only operation

7.1 Clarify what the requirement might mean

“Cross monitoring and listening” is not a single standardized feature name. It could mean passive observation of an existing bus, two controllers listening to each other on a bench bus, comparing one node’s transmitted messages against an independent logger, or receiving traffic from multiple CAN channels and comparing timing/content.

The safest first implementation is passive monitoring of an existing bus, followed by a separate bench test exchanging frames between two controlled nodes. Do not inject test traffic into a live control network without explicit authorization and a defined procedure.

7.2 Normal mode versus listen-only

In normal mode, a CAN interface can transmit and take part in acknowledgement and error signaling according to its controller’s behavior. In listen-only/silent mode, a properly configured controller monitors frames without transmitting acknowledgements or error flags. This is appropriate when the monitor must not influence the network.

A passive observer does not supply an ACK. If the target network relies on the monitor as its only potential acknowledger—which is abnormal or misconfigured—the sender may retransmit or report errors. A live network should have its own active receiving nodes. Validate listen-only semantics in the specific MCU/controller/tool documentation; names such as “silent”, “monitor”, “bus monitor”, and “listen-only” can differ between vendors.

7.3 Implementation outline

1. Configure the correct peripheral clock and CAN-FD nominal/data bit timing.
2. Set controller mode to listen-only/silent for passive capture.
3. Configure permissive filters first, then verify frames are arriving.
4. Enable RX FIFO/queue and receive notifications/interrupts.
5. In the interrupt path, read or enqueue frames efficiently; do not block on slow UART/USB/console output.

6. In the main loop/task, add timestamps, frame metadata, and loss/error statistics.
7. Export to a structured log (CSV or documented text format) or a PC capture tool.
8. Confirm the monitor is not transmitting by checking controller mode/status and, where possible, using an independent analyzer.
9. Run for an interval representative of expected traffic; check FIFO overrun, dropped frames, resets, and timestamp drift.

7.4 Receive pipeline and overflow

A receive interrupt should do the minimum necessary: acknowledge the correct interrupt cause, drain/read frames from the peripheral FIFO or queue according to the vendor API, copy them to a bounded ring buffer, update counters, and return. The consumer can format or decode messages later. Avoid dynamic allocation in the high-rate receive path unless runtime and timing guarantees are understood.

Track at least frames received by format/type, FIFO overrun/software queue overflow, controller errors and error state, bus-off events and recovery attempts, maximum queue occupancy if feasible, and logging failures/dropped log records. A log that silently drops frames is worse than one that reports its limitations.

7.5 Comparing two observations

For cross-monitoring, collect logs from the target controller and an independent analyzer or second receiver. Normalize timestamp origins if comparing timing. Compare identifier, frame format, BRS, payload length/data, frame ordering, and timing tolerance. Timestamp precision and timestamp point can differ between devices; a small offset does not automatically mean traffic differs.

Useful test matrix:

- A known periodic frame, if available, to compare expected versus measured period.
- A CAN-FD frame with BRS disabled.
- A CAN-FD frame with BRS enabled, if both nodes and setup support it.
- A payload larger than 8 bytes to prove CAN-FD handling.
- Standard and extended IDs if required.
- Traffic bursts to exercise FIFO limits.

- Error/bus-off observation using controlled fault injection only on an isolated test network.

7.6 Security and safety boundary

CAN is a shared control network, not a security boundary. Do not transmit crafted frames, change termination, or disconnect nodes on a live vehicle/industrial control bus as exploratory testing. Start with isolated hardware and explicit authorization. For passive observation, use listen-only mode and a physically appropriate interface. Monitoring does not decrypt, authenticate, or prove that message contents are trustworthy.

8. Bench validation and troubleshooting

8.1 Staged proof plan

- **Stage A — Power only.** Acceptance: correct rails, acceptable current, no abnormal heating, clean reset.
- **Stage B — MCU alive.** Acceptance: debugger attaches, firmware flashes, heartbeat works, build is reproducible.
- **Stage C — Controller configured.** Acceptance: correct peripheral clock/timing verified; no unexpected initialization error.
- **Stage D — Two-node bench CAN-FD.** Acceptance: controlled nodes exchange frames at known nominal/data bitrates, including payload over 8 bytes. Independent analyzer confirms format and payload.
- **Stage E — Passive monitor.** Acceptance: listen-only monitor captures known frames while not transmitting; counts/logs match reference analyzer within known timestamp/tool limits.
- **Stage F — Stress and recovery.** Acceptance: bursts do not silently lose frames; loss counters work; restart and bus-off handling are documented; firmware recovers safely after controlled reset.
- **Stage G — Target network.** Only after previous stages pass and network owner approves. Record target bit timing/topology and start in passive mode.

8.2 Diagnostic table

Symptom	Checks
No frames received	CANH/CANL wiring and ground/ reference, transceiver enable/

Symptom	Checks
	standby, peripheral pin mux, filters, bus activity, nominal timing, controller mode, actual CAN-FD support.
Classical CAN works but CAN-FD does not	CAN-FD capability of controller/transceiver/analyzer, FD acceptance setting, BRS/data timing, driver/API configuration, Classical-only monitor mode.
Frames work without BRS but fail with BRS	Data-phase bitrate/timing, peripheral clock, transceiver speed capability, wiring/stubs, topology, interface limitations.
Intermittent errors at high rate	Signal integrity, termination, cable/stubs, sample points, clock accuracy, supply noise, grounding, buffer overruns.
Monitor appears to disturb network	Confirm listen-only/silent mode, TX enable and tool settings; inspect auxiliary/automatic behavior.
Missing log entries but controller reports frames	Interrupt starvation, ring buffer size, blocking output, task priorities, queue ownership, overflow counters.
Bus-off	Wiring, bitrate mismatch, transceiver mode, lack of acknowledgers in normal mode, noise. Use documented recovery; do not blindly reset in a tight loop.

9. Decisions still required

The following must be resolved before production implementation or final BOM approval:

1. Exact MCU/board model and board revision.
2. Whether CAN-FD is integrated or an external controller is required.

3. Input supply voltage, source type, current budget, and environmental/transient exposure.
4. Number of CAN channels and their purpose.
5. Target nominal bitrate, data bitrate, BRS usage, standard/extended IDs, and known traffic specification.
6. Whether “cross monitoring” means passive capture, a second receiver, comparison of two channels, or something else.
7. Whether listen-only mode is mandatory on the real network.
8. Target platform/IDE/SDK and debugging probe.
9. Procurement budget, quantity, delivery deadline, and availability constraints.
10. Architecture overview and flowchart diagrams must be checked against this guide before treating any signal/block as confirmed.

These are not cosmetic details: they determine the MCU, transceiver, regulator, protection, connector, firmware architecture, and test plan.

10. Reference material

1. **Bosch Semiconductors, “CAN FD Protocol.”** Overview of CAN-FD payload, flexible bit rate, and protocol evolution. https://www.bosch-semiconductors.com/products/ip-modules/can-protocols/can-fd/
2. **CAN in Automation (CiA), “CAN FD - The basic idea.”** Arbitration/data phases, bit-rate switching, payload expansion, and frame concepts. https://www.can-cia.org/can-knowledge/can-fd-the-basic-idea
3. **STMicroelectronics, AN5348, “Introduction to FDCAN peripherals for STM32 MCUs,” Rev. 6, January 2026.** Protocol details and supported STM32 FDCAN peripherals. https://www.st.com/content/st_com/en/technical-documents/an5348-introduction-to-fdcan-peripherals-for-stm32-mcus-stmicroelectronics.html
4. **NXP, “S32K Automotive General-Purpose MCUs.”** Family-level information about integrated CAN-FD and tools; check exact part documents. https://www.nxp.com/products/processors-and-microcontrollers/s32-automotive-platform/s32k-auto-general-purpose-mcus:S32K-MCUS
5. **Microchip, “CAN FD MCUs.”** Overview of MCU/DSC families with integrated CAN-FD. https://www.microchip.com/en-us/products/interface-networking-connectivity/can/can-fd-mcus

6. **Microchip, MCP2517FD datasheet.** Example of an external CAN-FD controller using SPI and a separate transceiver. https://ww1.microchip.com/downloads/aemDocuments/documents/OTH/ProductDocuments/DataSheets/MCP2517FD-External-CAN-FD-Controller-with-SPI-Interface-20005688B.pdf
7. **Texas Instruments, TIDEP-01014 reference design.** CAN-FD example using a TCAN4550 external controller/interface. https://www.ti.com/tool/TIDEP-01014
8. **Repository reference assets:**
   Architecture overview: https://github.com/Chirag-R-Karanth/EL_5th_Sem/blob/main/Refrences/Architecture%20Overview.png
   Flowchart: https://github.com/Chirag-R-Karanth/EL_5th_Sem/blob/main/Refrences/Flowchart.png
   TODO: https://github.com/Chirag-R-Karanth/EL_5th_Sem/blob/main/todo.md

11. Quick review sheet

- **Architecture:** MCU CAN-FD controller -> CAN transceiver -> CANH/CANL. Controller handles protocol; transceiver handles electrical levels.
- **CAN-FD:** up to 64 data bytes; can use slower arbitration and faster data phase with BRS. Rates depend on network components and topology.
- **MCU:** verify CAN-FD peripheral, timing, clocks, filters, FIFO depth, pin mux, memory, SDK, errata, lifecycle.
- **Power:** derive input/protection/regulator values from real rails and loads; current-limit the first startup and test before connecting the bus.
- **Termination:** ordinarily 120 ohms at each end of a linear bus; two appear approximately 60 ohms in parallel with power off. Do not add termination blindly.
- **Passive monitoring:** use documented listen-only/silent mode and verify the monitor does not transmit. A passive observer does not ACK frames.
- **Development:** first get a reproducible build and heartbeat; then validate the controller; then two-node bench tests; then passive capture; only then the target network.
- **Procurement:** shortlist by exact electrical, timing, interface, lifecycle, and availability requirements—not part-family names alone.

- **Safety:** no exploratory transmission or wiring changes on a live control bus without authorization and a controlled test plan.

12. TODO coverage status

- [x] **Study the Microcontroller** — explained selection factors, documentation, and bring-up.
- [x] **Powering** — explained power-tree design, protection, regulator choices, and safe test sequence.
- [x] **Development environment** — explained toolchain, project structure, firmware layering, and reproducibility.
- [x] **Vendors to procure** — listed candidate vendor families and procurement/BOM criteria; final parts remain pending actual requirements.
- [x] **Enabling CAN-FD communication for cross monitoring and listening** — explained controller/transceiver architecture, CAN-FD fundamentals, listen-only mode, receive pipeline, and test plan.

Important meaning of “covered”: each topic has an in-depth explanation and an action plan. This does not mean physical hardware has been tested or the final part selection has been made.